

the warehouse. Every scanner at every depot along the way records when it arrived and when it left. At the end of the trip, you pull up the tracking number and the entire journey shows up on one page, in order.

Each scan along the way is called a span. A span is one unit of work (a database query, a call to an external API, a cache lookup) with a start time, an end time, and some labels describing what was happening. All the spans for a single request share the same trace ID. That shared ID is what lets a tool stitch them back into one story.

Tracing only really started to matter once one request stopped being handled by one program. Take a single checkout click that touches a frontend, an order service, a payment service, an inventory service, and three databases. Eight stops in total. A metric will tell you that click was slow. Only the trace will tell you that 92 percent of the time was spent waiting on the inventory database while six other things ran in under ten milliseconds each.

You do not need to memorise the rows. The point is that each pillar is built around a different question. Aim a question at the wrong pillar and you get an answer that is worse than if you had not asked at all.

Where each one falls short

Logs get out of hand quickly. A busy service can throw off several gigabytes a day, most of which is noise (startup messages, healthy responses, the same handful of patterns repeating). Storing all of it gets expensive enough that retention windows become a quarterly argument inside engineering. What stays for ninety days? What gets dropped after seven? Whether that one team's debug logs really need to live on hot storage?

Metrics have a different problem. They are good at telling you something is wrong and useless at telling you why. An error-rate spike is easy to see on a chart. The chart will not tell you which user triggered it, which code path failed, or whether it was the new

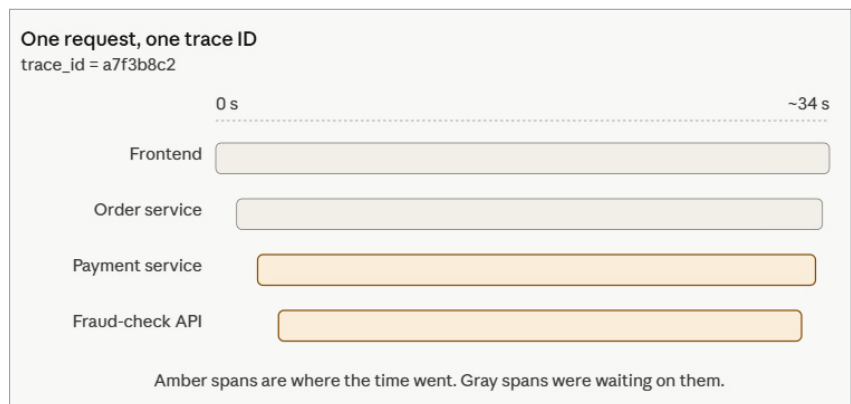


Figure 3: Trace ID

release from this morning. Averages cause their own trouble too. A response time of 200 milliseconds across all requests looks fine, until you find out one user in fifty is sitting through a four-second wait, and those are the users about to file a ticket.

Traces depend on every service in the chain doing its part. If one service skipped instrumentation, the trace breaks where that service handed off, and you cannot see where the time went. Traces also produce so much data that almost nobody keeps all of them. Most teams sample (one in every hundred requests, say, or only the failing ones), which means the exact trace you want is sometimes simply not in storage when you go looking.

What each one is actually for

Here is the part worth keeping in your head. The metric is what wakes you up. The trace tells you which corner of the system to look in. The logs are where the actual answer is hiding.

A short example to make that concrete. The on-call engineer's phone goes off at 4:47 in the morning. The alert says the checkout error rate has climbed from 0.3 percent to 5.8 percent in the last seven minutes. That is the metric doing its job. It does not say what is broken, only that something is.

The engineer opens the trace view, filters down to the failing requests from the last fifteen minutes, and finds almost all of them are stuck in the payment

service. Specifically, a span called *verify_fraud_score* is now averaging thirty-four seconds instead of 180 milliseconds. The trace has narrowed the whole incident down to one operation.

The last stop is the payment service logs from that window. The same line is repeating: timeout against an external fraud-check vendor's API. The fraud vendor is having an outage. Nothing the team built is broken. They just have to roll over to the backup provider.

That whole investigation took about eight minutes. Pull any one of the three signals out and the chain stops working. Without the metric, the alert never fires. Without the trace, the team is grep-ing through a haystack of log lines hoping the right one floats up. It's the log that points to the fix, but it is also useless without the other two doing their work first.

How the three work together

Engineers sometimes shorthand this as the 'what, where, why' pattern. A metric raises a flag (what is wrong), the trace narrows it down to a service or operation (where it is happening), and the log finishes the story (why). The order changes a little depending on the team and the type of incident, but the shape stays the same.

What nobody warns you about until it bites is that all three have to share context. If the log lines from the payment service and the trace that called